

Materia:	Programación IV ▾
Nivel:	4º Cuatrimestre ▾
Tipo de Examen:	Trabajo Práctico. Corresponde a Primer Parcial

TP #1 - Sala de juegos

Enunciado/s:

- Se debe realizar la aplicación “Sala de juegos”.
- La forma de corrección será por sprint de una semana.
- La aplicación debe permitir a nuestros usuarios (jugadores) medir sus capacidades cognitivas y motrices, debe ser simple de utilizar y permitir tener estadísticas de cada jugador y de cada juego.
- Los juegos deben guardar toda la información que sea importante para cada juego en particular.
- Solo se puede ingresar y jugar si es un usuario logueado o si se registra.
- Todos los juegos deben tener condiciones de finalización de victoria y de derrota claras.
- El TP cuenta con 4 sprints. El sprint #5 solo es obligatorio si el alumno debe entregar el TP en fecha de recuperatorios.

consideraciones

- No utilizar alert(), sino modales.

fin consideraciones

Contenido de la aplicación.

La “Sala de Juegos” tiene que contar con los siguientes puntos:

1. Aplicación frontend en Angular.
2. Servidor: Utilizar Supabase o Firebase.
3. Login y registro de usuarios: Autenticación y guardado de datos en base de datos.
4. Lógica de juegos:
 - a. Ahorcado.
 - b. Mayor o menor.
 - c. Preguntados.
 - d. Juego propio.
5. Juego propio: No deber ser ninguno de los siguientes:
 - a. Tatetí.

- b. Memotest.
- c. Piedra, papel o tijeras.
- 6. Sala de chat:
 - a. Debe ser una única sala que les permita a todos los usuarios registrados y logueados enviar mensajes y que se vean automáticamente (sin recargar la página).
- 7. Experiencia de usuario:
 - a. Las pantallas deben contar con diseño trabajado y uniforme a lo largo de la aplicación.
 - b. Navegación entre pantallas.
 - c. Información clara y completa al mostrar mensajes o realizar acciones.
 - d. Experiencia de usuario en los juegos. Tiempo de finalización / puntuación conseguida.
- 8. Diseño y estilos:
 - a. Utilización de bootstrap, primeNG o cualquier librería / paquete de estilos.
 - b. Utilización de animaciones (css y/o typescript).
 - c. Favicon de la aplicación.
- 9. Listados de resultados.
- 10. Presentación. Página “Quién soy”:
 - a. Datos personales del alumno.
 - b. Imagen del alumno.
 - c. Explicación del juego propio.

Entregas por sprint:

Sprint #1

- Creación del proyecto.
- Deploy en hosting (vercel, firebase, etc.).
- Componentes creados:
 - Login
 - Registro
 - Bienvenida / Home
 - Quién Soy
- Navegación entre componentes. Sin límites de accesibilidad.
- Funcionalidad - Componente Quién Soy:
 - Traer los datos del alumno desde la [api de github](#).
Ruta: <https://api.github.com/users/USERNAME>
Ejemplo: <https://api.github.com/users/torvalds>
 - Mostrar nombre del alumno, imagen de perfil y más datos.
 - Explicar de forma clara la elección del juego propio y cómo jugarlo.
- Implementar un [favicon](#) propio.

Sprint #2

- Funcionalidad - Componente Bienvenida / Home:
 - Tiene que ser el componente principal, desde este se podrán acceder a los diferentes juegos y listados.
 - Si el usuario NO está logueado, mostrar los botones de registro e inicio de sesión.
 - Si el usuario está logueado, mostrar su nombre de usuario y un botón para cerrar sesión.
 - Hasta no ver el tema “Guardias de ruta” o “Guards”, no es necesario bloquear los botones de los juegos, pero si ocultar los botones que no deberían verse (ejemplo, si estoy logueado, no debería ver el botón de registrarme). Una vez visto ese tema, bloquear la navegación a dichas rutas en el caso en el que sea necesario.
- Funcionalidad - Inicio de sesión:
 - Tiene que validar al usuario frente a supabase / firebase utilizando correo y contraseña.
 - En caso de que el inicio de sesión sea exitoso, navegar automáticamente al Home.
 - En caso de que el inicio de sesión no sea exitoso, mostrar un mensaje con el respectivo error.
 - La página de login debe contar con tres botones de inicio de sesión rápido, que le permitan a quién esté probando la aplicación ingresar automáticamente con usuarios previamente registrados para que las pruebas sean más ágiles.
- Funcionalidad: Registro.
 - Cuenta con un formulario que permite registrar a un usuario. Crea su cuenta en el sistema de autenticación **y** guarda sus datos en la base de datos. Nota: la contraseña **no** se guarda.
 - El usuario debe ingresar su correo, nombre, apellido, edad y contraseña.
 - Una vez cargados todos los datos, y el usuario se registra correctamente, se debe iniciar sesión con ese usuario y navegar automáticamente al Home.
 - Emitir mensaje si el usuario ya se encuentra registrado.

Sprint #3

- Juego: Ahorcado.
 - Deben mostrarse botones que simbolizen a todas las letras del abecedario. La entrada de datos es a través de botones, NO el teclado.
 - Al finalizar la partida, guardar en la base de datos: el usuario que jugó, junto con el tiempo de finalización, cantidad de letras seleccionadas, etc.
- Juego: Mayor o Mentor.
 - Se muestra una de una baraja de naipes. Se debe adivinar si la próxima carta va a ser un número mayor o un número menor.
 - Al finalizar la partida, guardar en la base de datos: el usuario que jugó, cantidad de cartas acertadas, etc.
- Sala de chat:

- Se debe mostrar el chat global para los usuarios logueados.
- Se debe permitir enviar un mensaje a la sala de chat.
- Al enviar un mensaje, este se guarda en la base de datos con el usuario que lo envió, el mensaje y la fecha de envío.
- Al guardar el mensaje, se debe mostrar en todos los clientes el nuevo mensaje automáticamente (se debe estar suscrito al servicio de base de datos en tiempo real).
- Se debe mostrar quién envía cada mensaje y a qué hora. El mensaje propio debe diferenciarse del resto.

Sprint #4

- Juego: Preguntados.
 - Debe obtener los datos de una [api](#).
 - Puede ser una api de preguntas o una api con información a la que luego se le agregue la funcionalidad de preguntados. Puede estar en inglés.
 - Las opciones de elección deben ser botones.
 - Al finalizar la partida, guardar en la base de datos: el usuario que jugó, cantidad de preguntas acertadas, etc.
- Crear el juego propio:
 - Agregar la descripción y reglas del mismo en la página quién soy.
 - Al finalizar la partida, guardar en la base de datos: el usuario que jugó y algún dato que mida su desempeño (puntaje, tiempo en finalizar, etc).
- Listados de resultados:
 - Crear la page Resultados.
 - Crear 4 tablas que detallen los resultados de los 4 juegos, mostrando el desempeño de cada jugador ordenado de mejor desempeño / puntaje a peor.

Sprint #5 - Recuperatorio

- Incorporar una encuesta:
 - Tiene que pedir los siguientes datos:
 - Nombre y apellido.
 - Edad, validar que sean mayores de 18 años y menores de 99 años.
 - Número de teléfono, validar que sean solo números y no más de 10 caracteres.
 - Mínimo 3 preguntas.
 - Utilizar distintos controles, textbox, checkbox, radiobutton, etc.
 - No se pueden repetir.
 - Tiene que contar con validaciones.
 - Todos los campos son requeridos.
 - Guardar las respuestas en la DB identificando el usuario.
- Incorporar una sección donde se puedan ver los resultados de las encuestas:
 - Solo pueden verlas usuarios marcados como administradores (utilizar guards).
- Añadir animaciones de transición de componentes.